OCR H446 / PAPER 1 / UNIT 1.2

Software and software development

CHAPTER CONTENTS

Select a topic to jump to its original worked pages.

1.2.1(a) / Operating-system responsibilities	4
1.2.1(b) / Paging, segmentation and virtual memory	8
1.2.1(c) / Interrupts and ISRs	12
1.2.1(d) / FCFS, SJF and round robin	16
1.2.1(d) / SRT and feedback queues	20
1.2.1(e-h) / OS types, firmware, drivers and VMs	24
1.2.2(a-c) / Applications, utilities and licensing	28
1.2.2(d,e) / Translators and compilation	32
1.2.2(f) / Linkers, loaders and libraries	36
1.2.3(a,b) / Software-development methodologies	40
1.2.3(c) / Write and follow algorithms	44
1.2.4(a,b) / Paradigms and procedural code	48
1.2.4(c,d) / Little Man Computer and addressing	52
1.2.4(e) / Object-oriented programming	56

H446/01 / REVISION

Understand it. Model it. Try it. Check it.

Use this chapter to understand the theory, follow worked models and practise independently.

FOUR SPECIFICATION FAMILIES

1.2.1 SYSTEMS SOFTWARE
OS, memory, interrupts, scheduling, firmware and VMs

1.2.2 APPLICATIONS
Utilities, translators, linking and licensing

1.2.3 DEVELOPMENT
Methodologies and algorithm tracing

1.2.4 LANGUAGES
Paradigms, LMC, addressing and OOP

FOUR PASSES FOR EVERY TOPIC

01 UNDERSTAND
Build a precise conceptual picture.

02 MODEL
Follow a fully worked trace or choice.

03 TRY
Answer original exam-style questions.

04 CHECK
Self-mark with separate indicative points.

Complete the questions before checking the indicative answers.

Explain relevant terminology clearly and show your working.

1.2 / SPECIFICATION CHECKLIST

Every examined 1.2 strand has a working example

Use the linked rows to jump to a topic. Content follows the accredited OCR H446 section 1.2 requirements, including all named scheduling methods and language paradigms.

SPEC	LEARNING CYCLE	PAGES
1.2.1(a)	Operating-system responsibilities	3–6
1.2.1(b)	Paging, segmentation and virtual memory	7–10
1.2.1(c)	Interrupts and ISRs	11–14
1.2.1(d)	FCFS, SJF and round robin	15–18
1.2.1(d)	SRT and feedback queues	19–22
1.2.1(e-h)	OS types, firmware, drivers and VMs	23–26
1.2.2(a-c)	Applications, utilities and licensing	27–30
1.2.2(d,e)	Translators and compilation	31–34
1.2.2(f)	Linkers, loaders and libraries	35–38
1.2.3(a,b)	Software-development methodologies	39–42
1.2.3(c)	Write and follow algorithms	43–46
1.2.4(a,b)	Paradigms and procedural code	47–50
1.2.4(c,d)	Little Man Computer and addressing	51–54
1.2.4(e)	Object-oriented programming	55–58

SCOPE BOUNDARY

1.2.4(c) LMC instructions use the standard OCR/LMC model; immediate, indirect and indexed examples are explicitly distinguished as generic pseudo-assembly.

Source: OCR H446 specification, Section 1.2. Use the links above to revise each requirement.

1.2.1(a) / BUILD THE MENTAL MODEL

The OS is the coordinator, not the application

A running program requests resources; the operating system coordinates who gets them and when.

THREE PIECES OF THE IDEA

PROCESS CONTROL

Allocate CPU time, change the running process and handle interrupts.

MEMORY CONTROL

Allocate and protect memory so processes do not overwrite one another.

FILES + DEVICES

Manage file permissions, file systems, I/O requests and hardware access.

MODEL / FOLLOW THE FLOW

Application → **System call** → **OS / scheduler** → **Hardware**

An application requests a file read; the OS mediates the operation rather than the application controlling the disk directly.

EXAM-READY STATEMENT

The OS is system software; the browser and backup tool remain separate programs using its services.

WORDS THAT EARN MARKS

- **Kernel** Core OS code
- **System call** OS service request
- **Scheduler** Chooses next task
- **Permissions** Restrict access

1.2.1(a) / STEP-BY-STEP WORKED EXAMPLE

Browser, backup and printer compete for one computer

Follow each change in order. A model is only useful when you can explain both what changed and why.

TRACE / DECISION TABLE

STEP	VALUE OR ACTION	REASON / INTERPRETATION
1. Request	Browser asks to load a file	OS checks permissions and file-system metadata.
2. Schedule	A backup runs in background	Scheduler allocates time so the browser remains responsive.
3. Memory	Each program needs RAM	Memory manager allocates/protects address spaces.
4. Device	Printer needs a particular driver	OS forwards request via driver and tracks its completion.
5. Return	OS reports outcome	Application receives data or an error, not raw hardware control.

TRANSFER IT TO AN EXAM ANSWER

The OS is system software; the browser and backup tool remain separate programs using its services.

Exam habit: state the condition, describe the mechanism, and link it to the specific outcome. Check calculations and intermediate values, not just the final result.

1.2.1(a) / INDEPENDENT EXAM PRACTICE

Use the model without copying it

Questions are original teacher-written practice, not OCR past-paper questions. Work in black pen; check the next page only afterwards.

BLACK PEN: ANSWER FIRST. THEN CHECK THE SEPARATE MARK POINTS.

QUESTION 1 | 4 MARKS

Explain two ways an operating system permits a browser and a background backup to run concurrently.

QUESTION 2 | 5 MARKS

A student application needs to print a confidential report. Explain the roles of the operating system, driver and file permissions.

1.2.1(a) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct, explain

Check method and vocabulary, not just the final number. These are teacher-written points, not an OCR mark scheme.

Q1 / 4 MARKS

- Scheduler shares CPU time between processes and permits progress [2].
- Memory management allocates/protects their memory spaces [2].

Q2 / 5 MARKS

- OS receives and manages the print request, possibly spooling it.
- Printer driver translates OS requests into hardware-specific commands.
- File/print access permissions restrict unauthorised users.
- The OS tracks I/O completion and reports failure to the application.

COMMON MISCONCEPTION

A multitasking OS does not require two CPU cores. Even one core can interleave tasks.

Green pen: Explain one OS function using a real request, action and outcome.

1.2.1(b) / BUILD THE MENTAL MODEL

A page is a fixed-size block; a segment has meaning

An address seen by a process need not equal a physical RAM location. The memory manager performs the mapping.

THREE PIECES OF THE IDEA

PAGING

Split virtual memory into equal-sized pages and physical RAM into equal-sized frames.

SEGMENTATION

Group a program into logical, variable-sized segments such as code, stack and data.

VIRTUAL MEMORY

Move pages/regions between RAM and secondary storage when needed; avoid frequent page faults.

MODEL / FOLLOW THE FLOW

Virtual page 3 → **Page table** → **Physical frame 7** → **RAM access**

Example: 1 KB pages; virtual address 3,100 uses page 3 plus offset 28. If page 3 maps to frame 7, physical address is 7,196.

EXAM-READY STATEMENT

Virtual memory can extend available address space, but access to secondary storage is much slower than RAM.

WORDS THAT EARN MARKS

Page	Fixed-size block	**Frame**	RAM block
Page fault	Page not resident	**Thrashing**	Excessive swapping

EXAM CHECK | INTERNAL AND EXTERNAL FRAGMENTATION

Paging may waste space inside a fixed-size allocated frame (internal fragmentation).

Segmentation may leave gaps between allocated regions (external fragmentation).

Do not confuse either form of fragmentation with a page fault or swapping.

1.2.1(b) / STEP-BY-STEP WORKED EXAMPLE

Translate 3,100 with 1,024-byte pages

Follow each change in order. A model is only useful when you can explain both what changed and why.

TRACE / DECISION TABLE

STEP	VALUE OR ACTION	REASON / INTERPRETATION
1. Divide	3,100 ÷ 1,024 = 3 rem 28	Virtual page number = 3, offset = 28.
2. Look up	Page table: page 3 → frame 7	Page table supplies the RAM frame number.
3. Multiply	7 × 1,024 = 7,168	Frame 7 starts at byte address 7,168.
4. Add offset	7,168 + 28 = 7,196	Physical address = 7,196; offset is unchanged.
5. Alternative	Page not resident?	Page fault: OS loads it from secondary storage, then retries.

TRANSFER IT TO AN EXAM ANSWER

Virtual memory can extend available address space, but access to secondary storage is much slower than RAM.

Exam habit: state the condition, describe the mechanism, and link it to the specific outcome. Check calculations and intermediate values, not just the final result.

SECOND WORKED MODEL | MEASURE THE WASTE

Paging: 2,500 bytes, 1,024-byte pages -> three frames, so 3,072 bytes allocated.

Internal waste = 3,072 - 2,500 = 572 bytes in the final frame.

Segmentation: free holes of 3 MB, 4 MB and 5 MB; a 6 MB segment cannot fit contiguously, even though 12 MB is free in total (external fragmentation).

A memory allocator may use different placement or compaction strategies.

1.2.1(b) / INDEPENDENT EXAM PRACTICE

Use the model without copying it

Questions are original teacher-written practice, not OCR past-paper questions. Work in black pen; check the next page only afterwards.

BLACK PEN: ANSWER FIRST. THEN CHECK THE SEPARATE MARK POINTS.

QUESTION 1 | 5 MARKS

Explain the difference between paging, segmentation and virtual memory. Refer to page tables or logical units.

QUESTION 2 | 5 MARKS

A system uses 512-byte pages. Virtual address 1,700 is on a page mapped to frame 9. Calculate the page, offset and physical address.

QUESTION 3 | 4 MARKS

Distinguish internal from external fragmentation. Use one example for each.

1.2.1(b) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct, explain

Check method and vocabulary, not just the final number. These are teacher-written points, not an OCR mark scheme.

Q1 / 5 MARKS

- Paging uses fixed-size pages and frames, linked by page tables.
- Segmentation uses variable-size logical program regions.
- Virtual memory allows non-resident data/pages to reside on secondary storage; access may trigger page faults.

Q2 / 5 MARKS

- 1,700 = 3 × 512 + 164: page 3, offset 164.
- Frame 9 base = 9 × 512 = 4,608.
- Physical address = 4,608 + 164 = 4,772.
- The offset is preserved through address translation.

COMMON MISCONCEPTION

OCR 1.2.1(b) virtual memory is NOT 1.1.3(d) virtual storage (network/cloud-backed files).

Green pen: Redo the address translation without looking, then explain the role of the page table.

Q3 / 4 MARKS | FRAGMENTATION

1 mark: internal = space unused within an allocated block or page/frame.
1 mark: example: 572 bytes wasted in a final 1,024-byte frame.
1 mark: external = free memory split into separated gaps.
1 mark: example: 3 + 4 + 5 MB holes cannot hold a contiguous 6 MB segment.

1.2.1(c) / BUILD THE MENTAL MODEL

An interrupt changes control, not the entire program

Hardware or software signals an event. The processor completes an appropriate instruction boundary, saves state and services it.

THREE PIECES OF THE IDEA

INTERRUPT

A signal that requests processor attention, such as I/O completion.

CONTEXT

Preserve program counter, registers and relevant processor state.

ISR

Interrupt service routine handles the event and returns control afterwards.

MODEL / FOLLOW THE FLOW



Simplified end-of-instruction model: an interrupt is recognised, the handler runs, then the original process resumes.

EXAM-READY STATEMENT

An interrupt is not the same as polling: polling repeatedly checks device status, while an interrupt signals attention.

WORDS THAT EARN MARKS

Interrupt	Attention signal	**ISR**	Event handler
Context	Saved CPU state	**Priority**	Service urgency

1.2.1(c) / STEP-BY-STEP WORKED EXAMPLE

Keyboard input arrives during an ADD instruction

Follow each change in order. A model is only useful when you can explain both what changed and why.

TRACE / DECISION TABLE

STEP	VALUE OR ACTION	REASON / INTERPRETATION
1. Normal	CPU fetches, decodes and executes	The running process retains its ordinary sequence.
2. Event	Keyboard controller raises interrupt	CPU detects a permitted pending request.
3. Preserve	Save PC/status/needed registers	Protect the interrupted process state.
4. Service	Run keyboard ISR	Read/acknowledge device data and record event.
5. Return	Restore state and resume	Next instruction continues as if the ISR had not altered it.

TRANSFER IT TO AN EXAM ANSWER

An interrupt is not the same as polling: polling repeatedly checks device status, while an interrupt signals attention.

Exam habit: state the condition, describe the mechanism, and link it to the specific outcome. Check calculations and intermediate values, not just the final result.

1.2.1(c) / INDEPENDENT EXAM PRACTICE

Use the model without copying it

Questions are original teacher-written practice, not OCR past-paper questions. Work in black pen; check the next page only afterwards.

BLACK PEN: ANSWER FIRST. THEN CHECK THE SEPARATE MARK POINTS.

QUESTION 1 | 5 MARKS

Describe the sequence of actions from an I/O interrupt to resuming the interrupted program.

QUESTION 2 | 4 MARKS

Explain why an interrupt-driven input device may be preferable to continuous polling.

1.2.1(c) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct, explain

Check method and vocabulary, not just the final number. These are teacher-written points, not an OCR mark scheme.

Q1 / 5 MARKS

- Interrupt is recognised by the processor at a suitable point.
- Current process context, such as PC and registers, is saved.
- Control transfers to the relevant ISR, which handles/acknowledges the event.
- Saved context is restored and the interrupted process resumes.

Q2 / 4 MARKS

- Polling uses repeated CPU checks even when no input is ready.
- Interrupt-based handling allows useful work between events.
- Interrupt dispatch/context saving has overhead; it is not always better for extremely frequent events.

COMMON MISCONCEPTION

The ISR is code; the interrupt is the event signal. A process is normally resumed, not started from its first instruction.

Green pen: Write the entire interrupt-handling sequence in numbered steps from memory.

1.2.1(d) / BUILD THE MENTAL MODEL

Same tasks, different waiting times

A scheduling timeline must state arrival times, burst lengths and whether pre-emption is permitted.

THREE PIECES OF THE IDEA

FCFS

Run in arrival order; simple but a long task can delay later jobs.

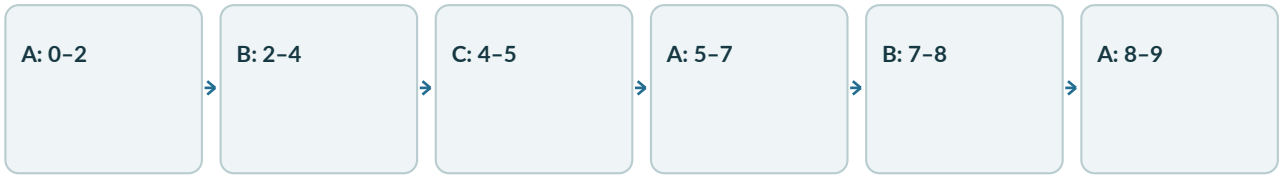
SJF

Choose shortest available total job; may leave large jobs waiting.

ROUND ROBIN

Use a time quantum, then rotate unfinished tasks to ready-queue tail.

MODEL / FOLLOW THE FLOW



Round robin example (all ready at t=0, quantum = 2): A needs 5, B needs 3 and C needs 1 time units.

EXAM-READY STATEMENT

For non-preemptive SJF, choose the shortest *available* task only when the CPU becomes free.

WORDS THAT EARN MARKS

Pre-emption Pause running job

Quantum CPU time slice

Waiting Ready but not run

Turnaround Finish minus arrival

1.2.1(d) / STEP-BY-STEP WORKED EXAMPLE

Calculate the round-robin completion and waiting times

Follow each change in order. A model is only useful when you can explain both what changed and why.

TRACE / DECISION TABLE

STEP	VALUE OR ACTION	REASON / INTERPRETATION
1. Input	A:5; B:3; C:1; all arrive t=0	Ready queue begins A, B, C; quantum = 2.
2. Slice 1	A 0–2; B 2–4; C 4–5	A has 3 left; B 1; C completes at t=5.
3. Slice 2	A 5–7; B 7–8	A has 1 left; B completes at t=8.
4. Last	A 8–9	A completes at t=9.
5. Waiting	Completion − arrival − CPU time	A = 4; B = 5; C = 4; mean = 13/3 = 4.33.

TRANSFER IT TO AN EXAM ANSWER

For non-preemptive SJF, choose the shortest *available* task only when the CPU becomes free.

Exam habit: state the condition, describe the mechanism, and link it to the specific outcome. Check calculations and intermediate values, not just the final result.

1.2.1(d) / INDEPENDENT EXAM PRACTICE

Use the model without copying it

Questions are original teacher-written practice, not OCR past-paper questions. Work in black pen; check the next page only afterwards.

BLACK PEN: ANSWER FIRST. THEN CHECK THE SEPARATE MARK POINTS.

QUESTION 1 | 5 MARKS

Using A=5, B=3, C=1 (all arriving at time 0), draw a round-robin trace with time quantum 2.

QUESTION 2 | 5 MARKS

Explain a strength and a weakness of each of FCFS and SJF compared with round robin.

1.2.1(d) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct, explain

Check method and vocabulary, not just the final number. These are teacher-written points, not an OCR mark scheme.

Q1 / 5 MARKS

- A 0–2, B 2–4, C 4–5, A 5–7, B 7–8, A 8–9.
- Completion times: A=9, B=8, C=5.
- Waiting times: A=4, B=5, C=4.

Q2 / 5 MARKS

- FCFS is simple with little scheduling overhead; a long first job causes convoy delays.
- SJF can reduce average waiting time when burst lengths are known; long tasks may starve.
- Round robin is responsive/fair to interactive jobs, but frequent switches add overhead.

COMMON MISCONCEPTION

Waiting time is not the same as turnaround time. Use waiting = completion − arrival − CPU burst.

Green pen: Draw a numbered timeline, check that the total span equals the sum of CPU bursts, and compute each completion.

1.2.1(d) / BUILD THE MENTAL MODEL

Pre-emption changes a running decision

Shortest remaining time (SRT) can replace the running task on a new arrival; multilevel feedback queues change priority based on behaviour.

THREE PIECES OF THE IDEA

SRT

At each arrival/completion, choose task with least remaining CPU time.

MLFQ

Several priority queues with differing time quanta; priorities can change as tasks use CPU.

TRADE-OFF

Excellent short-job response can cause starvation without suitable ageing or promotion.

MODEL / FOLLOW THE FLOW



SRT: A arrives at 0 (8 units), B at 1 (4 units), C at 2 (2 units). Each new arrival is shorter than the remaining job.

EXAM-READY STATEMENT

A scheduling simulation needs a clear rule for ties; these jobs avoid ties at decision points.

WORDS THAT EARN MARKS

SRT Least time remaining

MLFQ Feedback priorities

Ageing Raise waiting task

Starvation Waits indefinitely

1.2.1(d) / STEP-BY-STEP WORKED EXAMPLE

SRT: finish times for staggered arrivals

Follow each change in order. A model is only useful when you can explain both what changed and why.

TRACE / DECISION TABLE

STEP	VALUE OR ACTION	REASON / INTERPRETATION
t=0	Run A for 1 unit	A has 7 remaining when B arrives.
t=1	Run B for 1 unit	B has 3 remaining when C arrives.
t=2	Run C for 2 units	C completes at t=4; B is now shortest.
t=4	Run B for 3 units	B completes at t=7; only A remains.
t=7	Run A for 7 units	A completes at t=14.
Waiting	A=14−0−8=6; B=7−1−4=2; C=4−2−2=0	Mean waiting time = 8/3 ≈ 2.67.

TRANSFER IT TO AN EXAM ANSWER

A scheduling simulation needs a clear rule for ties; these jobs avoid ties at decision points.

Exam habit: state the condition, describe the mechanism, and link it to the specific outcome. Check calculations and intermediate values, not just the final result.

1.2.1(d) / INDEPENDENT EXAM PRACTICE

Use the model without copying it

Questions are original teacher-written practice, not OCR past-paper questions. Work in black pen; check the next page only afterwards.

BLACK PEN: ANSWER FIRST. THEN CHECK THE SEPARATE MARK POINTS.

QUESTION 1 | 5 MARKS

Draw the SRT timeline for A(0,8), B(1,4), C(2,2), where pairs represent (arrival, CPU time).

QUESTION 2 | 5 MARKS

Explain how a multilevel feedback queue might help short interactive jobs while risking starvation of a CPU-bound job.

1.2.1(d) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct, explain

Check method and vocabulary, not just the final number. These are teacher-written points, not an OCR mark scheme.

Q1 / 5 MARKS

- A 0–1; B 1–2; C 2–4; B 4–7; A 7–14.
- Completion A=14, B=7, C=4.
- Waiting A=6, B=2, C=0.

Q2 / 5 MARKS

- High-priority queue permits quicker response/shorter quantum for interactive bursts.
- CPU-intensive tasks may be demoted to lower-priority queues.
- A stream of high-priority jobs can delay low-priority tasks indefinitely.
- Ageing or periodic priority boosts can mitigate starvation.

COMMON MISCONCEPTION

Shortest remaining time compares *remaining* burst length, not original burst length or job ID.

Green pen: At each new arrival, annotate the remaining burst time beside every ready task.

1.2.1(e-h) / BUILD THE MENTAL MODEL

Know whether the task is time-critical or hardware-specific

Choose the operating-system model by what the system must guarantee, then separate firmware, device-driver and virtual-machine roles.

THREE PIECES OF THE IDEA

OS TYPES

Distributed, embedded, multitasking, multi-user and real-time describe different requirements.

BOOT + DEVICE

BIOS/UEFI initialises hardware; device drivers expose device-specific functionality.

VIRTUAL MACHINE

Software emulates a whole system or runs intermediate code within a managed execution environment.

MODEL / FOLLOW THE FLOW

Power on → **BIOS / UEFI** → **Boot loader / OS** → **Driver / service**

At startup, firmware performs initialisation and begins booting; it does not replace the OS or a printer driver.

EXAM-READY STATEMENT

A Java-style bytecode virtual machine and an OS guest virtual machine are two different examples of software providing machine functionality.

WORDS THAT EARN MARKS

Real-time Deadline matters

BIOS Startup firmware

Driver Device interface

VM Software machine

1.2.1(e-h) / STEP-BY-STEP WORKED EXAMPLE

Choose software for three real requirements

Follow each change in order. A model is only useful when you can explain both what changed and why.

TRACE / DECISION TABLE

STEP	VALUE OR ACTION	REASON / INTERPRETATION
Vehicle	Airbag response has hard deadline	Real-time system; predictability is crucial, not just average speed.
School	Many users share a server	Multi-user OS offers isolation, permissions and concurrent service.
Cluster	Machines cooperate over a network	Distributed OS/software coordinates work across multiple computers.
Sensor	Single-purpose environmental logger	Embedded OS may be compact and dedicated to the device.
Start-up	Firmware initialises and checks hardware	BIOS/UEFI transfers control towards boot-loading OS.
Printer	OS asks driver to print a page	Driver translates generic request to device protocol.
Testing	Run guest OS on host hypervisor	VM isolates a different system; separate from bytecode VM.

TRANSFER IT TO AN EXAM ANSWER

A Java-style bytecode virtual machine and an OS guest virtual machine are two different examples of software providing machine functionality.

Exam habit: state the condition, describe the mechanism, and link it to the specific outcome. Check calculations and intermediate values, not just the final result.

1.2.1(e-h) / INDEPENDENT EXAM PRACTICE

Use the model without copying it

Questions are original teacher-written practice, not OCR past-paper questions. Work in black pen; check the next page only afterwards.

BLACK PEN: ANSWER FIRST. THEN CHECK THE SEPARATE MARK POINTS.

QUESTION 1 | 5 MARKS

Justify a suitable OS classification for an airbag controller and a school file server.

QUESTION 2 | 6 MARKS

Distinguish BIOS/UEFI, a device driver, an OS-hosting virtual machine and a bytecode virtual machine.

1.2.1(e-h) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct, explain

Check method and vocabulary, not just the final number. These are teacher-written points, not an OCR mark scheme.

Q1 / 5 MARKS

- Airbag: real-time/embedded; deadline and predictable response are essential.
- School server: multi-user; must manage accounts, file permissions and simultaneous access.
- Other OS descriptors can overlap; justify the actual requirement.

Q2 / 6 MARKS

- BIOS/UEFI: firmware initialises hardware and starts boot process.
- Driver: device-specific software translates I/O requests.
- System VM: guest OS executes on virtualised hardware via hypervisor.
- Bytecode VM: interprets/executes intermediate code for a runtime environment.

COMMON MISCONCEPTION

Real-time does not mean fastest possible; it means meeting specified timing constraints reliably.

Green pen: Replace a vague phrase like “faster” with an explicit requirement such as response deadline or isolation.

1.2.2(a-c) / BUILD THE MENTAL MODEL

Select software because its properties meet the problem

A good justification begins with the user need, then links a specific product property to an outcome.

THREE PIECES OF THE IDEA

APPLICATION

Performs a user task: bespoke or off-the-shelf; assess fitness, support and cost.

UTILITY

Maintains/protects/optimises a system: backup, compression, anti-malware, encryption.

SOURCE MODEL

Open source offers source access under licence; closed source withholds source. Neither guarantees zero cost.

MODEL / FOLLOW THE FLOW

User requirement → **Required features** → **Constraints** → **Justified choice**

A school needs an accessible timetable tool: choose by features, data protection, support and maintenance, not by label alone.

EXAM-READY STATEMENT

Open source and proprietary describe licensing/source availability. They are not synonyms for secure/insecure or free/paid.

WORDS THAT EARN MARKS

Bespoke	Custom-made	**Utility**	Maintains system
Open source	Source available	**Licence**	Usage conditions

1.2.2(a-c) / STEP-BY-STEP WORKED EXAMPLE

Select a timetable system for a school

Follow each change in order. A model is only useful when you can explain both what changed and why.

TRACE / DECISION TABLE

STEP	VALUE OR ACTION	REASON / INTERPRETATION
1. Need	Staff, rooms, accessibility	List must-have tasks and accommodations.
2. Options	Bespoke or existing package	Bespoke can fit workflow; existing package may deploy faster.
3. Security	Pupil data is sensitive	Check updates, access control, audit and compliance.
4. Support	Who maintains it?	Compare support commitments and long-term costs.
5. Decision	Explain choice against criteria	A defensible answer links features to school needs.

TRANSFER IT TO AN EXAM ANSWER

Open source and proprietary describe licensing/source availability. They are not synonyms for secure/insecure or free/paid.

Exam habit: state the condition, describe the mechanism, and link it to the specific outcome. Check calculations and intermediate values, not just the final result.

1.2.2(a-c) / INDEPENDENT EXAM PRACTICE

Use the model without copying it

Questions are original teacher-written practice, not OCR past-paper questions. Work in black pen; check the next page only afterwards.

BLACK PEN: ANSWER FIRST. THEN CHECK THE SEPARATE MARK POINTS.

QUESTION 1 | 6 MARKS

Evaluate bespoke and off-the-shelf software for a school timetabling system. Refer to at least three criteria.

QUESTION 2 | 4 MARKS

Explain why a backup utility and an open-source software licence solve different problems.

1.2.2(a-c) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct, explain

Check method and vocabulary, not just the final number. These are teacher-written points, not an OCR mark scheme.

Q1 / 6 MARKS

- Bespoke can fit unusual school requirements but costs more to develop/test/support.
- Off-the-shelf deploys faster and can be cheaper but may require compromise.
- Justify with requirements such as accessibility, integration, security and ongoing support.
- A reasoned conclusion earns credit when anchored to stated needs.

Q2 / 4 MARKS

- Backup utility creates recoverable copies of files against loss.
- Open-source licence grants permission to inspect/modify/redistribute under stated conditions.
- One is software functionality; the other is a legal permission model.

COMMON MISCONCEPTION

A backup is not the same as a synchronised copy; deletion can propagate through sync, so recovery versions matter.

Green pen: Rewrite a generic preference as “This option is suitable because [feature] enables [specific user outcome].”

1.2.2(d,e) / BUILD THE MENTAL MODEL

A compiler checks structure before machine instructions run

Interpreters, compilers and assemblers translate different inputs or use different execution strategies; compilation itself has stages.

THREE PIECES OF THE IDEA

COMPILER

Translates a program (typically whole source unit) to target code before running it.

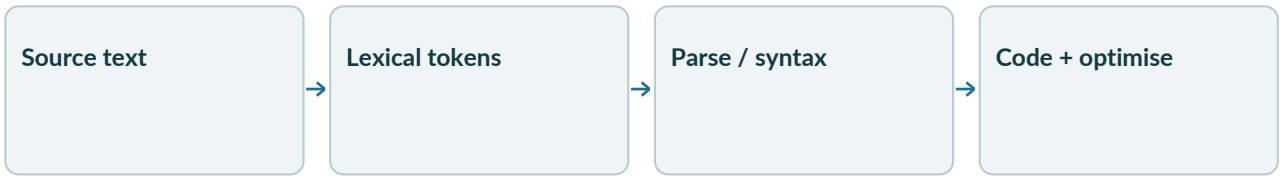
INTERPRETER

Executes through translation at run time; may report faults as reached.

ASSEMBLER

Maps assembly-language mnemonics and labels to machine-code representation.

MODEL / FOLLOW THE FLOW



Example: total = price + 2 becomes identifier, assignment, identifier, operator, integer tokens; then grammar and code checks.

EXAM-READY STATEMENT

A syntax error is found during analysis; a valid program can still contain semantic/logic errors.

WORDS THAT EARN MARKS

Lexical Tokens from text

Syntax Grammar checks

Code gen Produces target

Optimise Improve code

1.2.2(d,e) / STEP-BY-STEP WORKED EXAMPLE

Process a short assignment

Follow each change in order. A model is only useful when you can explain both what changed and why.

TRACE / DECISION TABLE

STEP	VALUE OR ACTION	REASON / INTERPRETATION
Source	total = price + 2	Program contains identifiers, assignment, addition, literal.
Lexical	ID(total), =, ID(price), +, INT(2)	Scanner groups characters into tokens.
Syntax	assignment → ID = expression	Parser checks valid arrangement/grammar.
Code gen	LOAD price; ADD 2; STORE total	Illustrative low-level instructions, not literal LMC assembly.
Optimise	Reduce redundant loads where safe	Must preserve observable program meaning.

TRANSFER IT TO AN EXAM ANSWER

A syntax error is found during analysis; a valid program can still contain semantic/logic errors.

Exam habit: state the condition, describe the mechanism, and link it to the specific outcome. Check calculations and intermediate values, not just the final result.

1.2.2(d,e) / INDEPENDENT EXAM PRACTICE

Use the model without copying it

Questions are original teacher-written practice, not OCR past-paper questions. Work in black pen; check the next page only afterwards.

BLACK PEN: ANSWER FIRST. THEN CHECK THE SEPARATE MARK POINTS.

QUESTION 1 | 6 MARKS

Describe lexical analysis, syntax analysis, code generation and optimisation for a compiler.

QUESTION 2 | 4 MARKS

Compare compiler, interpreter and assembler, with an appropriate input or use for each.

1.2.2(d,e) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct, explain

Check method and vocabulary, not just the final number. These are teacher-written points, not an OCR mark scheme.

Q1 / 6 MARKS

- Lexical analysis creates tokens and can discard irrelevant whitespace/comments.
- Syntax analysis checks token order against language grammar.
- Code generation produces target/intermediate instructions.
- Optimisation improves efficiency without changing intended behaviour.

Q2 / 4 MARKS

- Compiler translates high-level source to output code ahead of execution.
- Interpreter translates/executes at runtime (implementation details vary).
- Assembler converts symbolic assembly mnemonics to machine code.

COMMON MISCONCEPTION

Compiled does not mean bug-free. Translation can succeed while the algorithm still produces the wrong answer.

Green pen: Take one expression and underline which information belongs to lexical, syntax and code-generation stages.

1.2.2(f) / BUILD THE MENTAL MODEL

Compiled objects still need to become a running program

A compiler may create object code, but a linker resolves external references and a loader prepares the executable for execution.

THREE PIECES OF THE IDEA

LIBRARY

Reusable prewritten code providing functions, classes or services.

LINKER

Combines object modules and resolves symbols/references to libraries.

LOADER

Places executable in memory and prepares its process for running.

MODEL / FOLLOW THE FLOW



Two compiled source modules can call one library function; linking connects the references; loading starts the process.

EXAM-READY STATEMENT

Static and dynamic linking are different approaches: a library can be incorporated at build time or loaded/linked at run time.

WORDS THAT EARN MARKS

Object file Compiled module

Linker Resolves references

Symbol Named reference

Loader Prepares execution

1.2.2(f) / STEP-BY-STEP WORKED EXAMPLE

Program uses a shared graph library

Follow each change in order. A model is only useful when you can explain both what changed and why.

TRACE / DECISION TABLE

STEP	VALUE OR ACTION	REASON / INTERPRETATION
1. Compile	app.c → app.o	Object code may still refer to external drawGraph().
2. Reference	Unresolved drawGraph symbol	External routine implementation not in app.o.
3. Link	Link app.o to appropriate library	Resolve references; produce linked executable or dynamic references.
4. Load	OS loader maps code/data into memory	Arrange address space and required shared modules.
5. Run	Control transfers to entry point	Program uses library routine when required.

TRANSFER IT TO AN EXAM ANSWER

Static and dynamic linking are different approaches: a library can be incorporated at build time or loaded/linked at run time.

Exam habit: state the condition, describe the mechanism, and link it to the specific outcome. Check calculations and intermediate values, not just the final result.

1.2.2(f) / INDEPENDENT EXAM PRACTICE

Use the model without copying it

Questions are original teacher-written practice, not OCR past-paper questions. Work in black pen; check the next page only afterwards.

BLACK PEN: ANSWER FIRST. THEN CHECK THE SEPARATE MARK POINTS.

QUESTION 1 | 5 MARKS

Explain what a linker does when a program calls a function stored in a library.

QUESTION 2 | 5 MARKS

Explain two advantages and one limitation of using libraries, and outline the loader's role.

1.2.2(f) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct, explain

Check method and vocabulary, not just the final number. These are teacher-written points, not an OCR mark scheme.

Q1 / 5 MARKS

- Compiled modules can contain unresolved external symbols.
- Linker matches symbols to module/library definitions.
- It combines code/metadata and creates appropriate addresses or relocations.
- Output is a linked executable or suitable dynamic-link structure.

Q2 / 5 MARKS

- Libraries provide tested, reusable functionality and reduce duplicated development.
- Updates/standardisation can improve maintainability.
- Version conflicts, dependencies or security flaws can be introduced.
- Loader maps executable and dependencies into memory and prepares execution.

COMMON MISCONCEPTION

A linker does not itself run the program. A loader does not perform the compiler's lexical analysis.

Green pen: Label where each of source, object, executable and running process exists in the pipeline.

1.2.3(a,b) / BUILD THE MENTAL MODEL

Choose a method by risk, certainty and feedback

A methodology structures change and validation; the best choice depends on stability, risk, user involvement and deadlines.

THREE PIECES OF THE IDEA

WATERFALL

Sequential staged work; useful with stable requirements and approval gates.

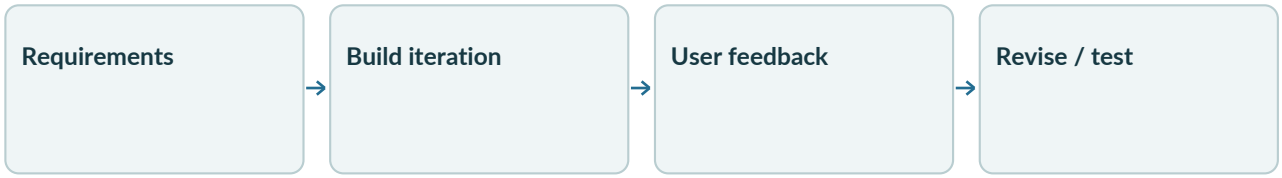
AGILE / XP

Incremental delivery; XP stresses practices such as pair programming and frequent tests.

SPIRAL / RAD

Spiral explicitly addresses risk across cycles; RAD prioritises quick prototypes and user feedback.

MODEL / FOLLOW THE FLOW



An iterative product improves after each usable increment. Waterfall may enforce sequential stage sign-off instead.

EXAM-READY STATEMENT

Agile is not “no documentation”; waterfall is not “no testing”. Explain when each trade-off matters.

WORDS THAT EARN MARKS

- **Iteration** Repeat build cycle
- **Prototype** Early model
- **Spiral** Risk-led cycle
- **XP** Agile practices

1.2.3(a,b) / STEP-BY-STEP WORKED EXAMPLE

Choose a method for two contrasting projects

Follow each change in order. A model is only useful when you can explain both what changed and why.

TRACE / DECISION TABLE

STEP	VALUE OR ACTION	REASON / INTERPRETATION
A: payroll	Fixed legal calculation rules	Structured documentation and test stages are important.
Trade-off	Waterfall offers stage control	Late changes are costly; users see final product relatively late.
B: school app	Users uncertain about interface	RAD/agile prototypes can elicit precise feedback early.
Trade-off	Iterative approach enables changes	Scope and quality must still be controlled by tests.
High risk	Unproven technical integration	Spiral builds risk assessment into each cycle.
Team culture	Pair review and automated tests	XP practices can strengthen frequent releases.

TRANSFER IT TO AN EXAM ANSWER

Agile is not “no documentation”; waterfall is not “no testing”. Explain when each trade-off matters.

Exam habit: state the condition, describe the mechanism, and link it to the specific outcome. Check calculations and intermediate values, not just the final result.

1.2.3(a,b) / INDEPENDENT EXAM PRACTICE

Use the model without copying it

Questions are original teacher-written practice, not OCR past-paper questions. Work in black pen; check the next page only afterwards.

BLACK PEN: ANSWER FIRST. THEN CHECK THE SEPARATE MARK POINTS.

QUESTION 1 | 6 MARKS

Evaluate waterfall and agile approaches for a school reporting tool with changing teacher requirements.

QUESTION 2 | 5 MARKS

Explain why the spiral model may be preferable to RAD for safety-critical control software.

1.2.3(a,b) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct, explain

Check method and vocabulary, not just the final number. These are teacher-written points, not an OCR mark scheme.

Q1 / 6 MARKS

- Waterfall offers up-front planning, staged review and predictable documentation.
- It may handle late changes poorly and delay user feedback.
- Agile enables incremental release and feedback when requirements evolve.
- It requires active user involvement and control of scope/test quality.
- Conclude using the actual school context.

Q2 / 5 MARKS

- Spiral includes explicit risk identification, assessment and mitigation each cycle.
- High-risk design decisions can be explored before major commitment.
- RAD rapidly develops prototypes but speed alone is not a safety guarantee.
- Safety still demands verification, documentation and controlled testing.

COMMON MISCONCEPTION

Extreme programming is a named agile approach; it is not simply writing code very quickly.

Green pen: Write one “suitable because” and one “limited because” statement for each method.

1.2.3(c) / BUILD THE MENTAL MODEL

Trace the condition before changing a variable

A trace table records state after clearly defined statements. Distinguish loop-entry tests from updates inside the body.

THREE PIECES OF THE IDEA

SPECIFY

State inputs, outputs and required behaviour before writing steps.

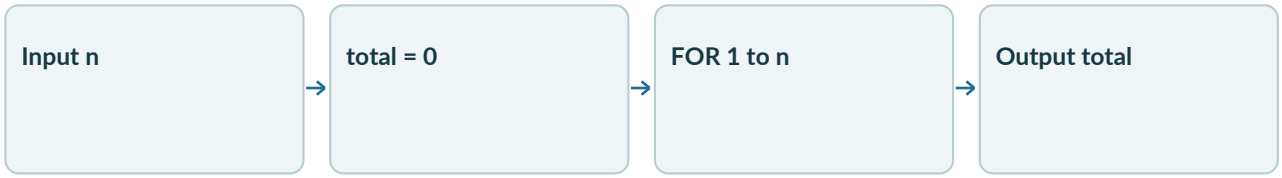
TRACE

Record each iteration's values and branch outcome systematically.

VERIFY

Test normal, boundary and invalid cases to expose faults.

MODEL / FOLLOW THE FLOW



Example algorithm accumulates 1+2+...+n; n=4 should output 10. The accumulator changes after each addition.

EXAM-READY STATEMENT

At each completed pass, total equals the sum of all integers already processed. This is an invariant useful for explanation.

WORDS THAT EARN MARKS

Trace	State by step	**Invariant**	Maintained truth
Boundary	Edge input	**Dry run**	Manual execution

1.2.3(c) / STEP-BY-STEP WORKED EXAMPLE

Dry-run a running total for n = 4

Follow each change in order. A model is only useful when you can explain both what changed and why.

TRACE / DECISION TABLE

STEP	VALUE OR ACTION	REASON / INTERPRETATION
Initial	n = 4; total = 0	No iteration has run.
i = 1	total = 0 + 1 = 1	First pass.
i = 2	total = 1 + 2 = 3	Second pass.
i = 3	total = 3 + 3 = 6	Third pass.
i = 4	total = 6 + 4 = 10	Fourth pass.
After	Output 10	The loop has completed its inclusive upper bound.

TRANSFER IT TO AN EXAM ANSWER

At each completed pass, total equals the sum of all integers already processed. This is an invariant useful for explanation.

Exam habit: state the condition, describe the mechanism, and link it to the specific outcome. Check calculations and intermediate values, not just the final result.

1.2.3(c) / INDEPENDENT EXAM PRACTICE

Use the model without copying it

Questions are original teacher-written practice, not OCR past-paper questions. Work in black pen; check the next page only afterwards.

BLACK PEN: ANSWER FIRST. THEN CHECK THE SEPARATE MARK POINTS.

QUESTION 1 | 5 MARKS

Trace total ← 0; FOR i FROM 1 TO 5: total ← total + i. Give total after every iteration and the output.

QUESTION 2 | 5 MARKS

Design an algorithm that inputs three integers and outputs the largest. State a boundary test.

1.2.3(c) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct, explain

Check method and vocabulary, not just the final number. These are teacher-written points, not an OCR mark scheme.

Q1 / 5 MARKS

- Totals after i=1,2,3,4,5 are 1,3,6,10,15.
- Final output is 15, assuming an inclusive loop 1..5.

Q2 / 5 MARKS

- Read a,b,c; set largest ← a; if b > largest then largest ← b; if c > largest then largest ← c; output largest.
- Test equal values, e.g. (3,3,3), and negatives, e.g. (−5,−2,−9), as relevant edge cases.
- A diagram or equivalent correct pseudocode may earn credit.

COMMON MISCONCEPTION

Updating total before recording it changes the meaning of a trace row. State exactly when values are measured.

Green pen: Add a trace table with columns for iteration number and accumulator after the loop body.

1.2.4(a,b) / BUILD THE MENTAL MODEL

Different styles organise the same computation

A paradigm is a programming approach. Procedural programs organise operations into procedures/functions with explicit control flow.

THREE PIECES OF THE IDEA

PROCEDURAL

Step-by-step algorithms, variables, loops, parameterised routines.

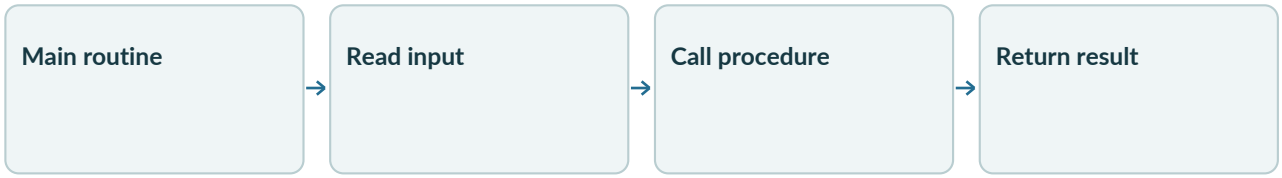
OBJECT-ORIENTED

Model objects containing state and behaviour, with encapsulation and reuse.

DECLARATIVE / FUNCTIONAL

Describe desired results or compose functions with fewer mutable side effects (different styles).

MODEL / FOLLOW THE FLOW



A procedural program decomposes a calculation into named routines. Data is passed by parameters and/or stored in state.

EXAM-READY STATEMENT

A procedure performs an action; a function returns a value in conventional terminology. Language implementations differ.

WORDS THAT EARN MARKS

Paradigm Programming style

Procedure Named action

Parameter Supplied value

State Current values

1.2.4(a,b) / STEP-BY-STEP WORKED EXAMPLE

Break a bill calculator into routines

Follow each change in order. A model is only useful when you can explain both what changed and why.

TRACE / DECISION TABLE

STEP	VALUE OR ACTION	REASON / INTERPRETATION
Main	get unit price and quantity	Input handled before calculation.
Call	subtotal = cost(price,qty)	Function receives two arguments.
Inside	RETURN price × qty	Return value calculated, not printed inside function.
Process	vat = subtotal × 0.20	Separate stage can be tested independently.
Output	print subtotal + vat	Main routine handles presentation.

TRANSFER IT TO AN EXAM ANSWER

A procedure performs an action; a function returns a value in conventional terminology. Language implementations differ.

Exam habit: state the condition, describe the mechanism, and link it to the specific outcome. Check calculations and intermediate values, not just the final result.

1.2.4(a,b) / INDEPENDENT EXAM PRACTICE

Use the model without copying it

Questions are original teacher-written practice, not OCR past-paper questions. Work in black pen; check the next page only afterwards.

BLACK PEN: ANSWER FIRST. THEN CHECK THE SEPARATE MARK POINTS.

QUESTION 1 | 5 MARKS

Explain two advantages of procedural decomposition for a large program.

QUESTION 2 | 5 MARKS

Describe two differences between procedural and object-oriented approaches, with an appropriate example.

1.2.4(a,b) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct, explain

Check method and vocabulary, not just the final number. These are teacher-written points, not an OCR mark scheme.

Q1 / 5 MARKS

- Named procedures/functions reduce duplication and support reuse.
- Individual routines can be tested and maintained separately.
- Parameters clarify data transferred between routines.
- Clear control flow supports reasoning and debugging.

Q2 / 5 MARKS

- Procedural code organises operations and data flow into routines.
- OOP groups attributes and methods in objects/classes.
- Procedural example: calculateTax function; OOP example: Invoice object with total() and stored line items.

COMMON MISCONCEPTION

Functions are not automatically object-oriented. Both procedural and OO programs can use functions and parameters.

Green pen: Replace a lengthy monolithic procedure with two named subroutines and state each input/output.

1.2.4(c,d) / BUILD THE MENTAL MODEL

Separate the instruction from the value it refers to

LMC illustrates an accumulator machine using mailboxes. Addressing modes determine how an operand is obtained.

THREE PIECES OF THE IDEA

LMC

LDA, STA, ADD, SUB, INP, OUT, BRA, BRZ, BRP, HLT and DAT model basic operations.

ADDRESS

Direct uses the addressed location; immediate embeds a value; indirect reads a pointer.

INDEXED

Effective address combines a base address with an index register or offset.

MODEL / FOLLOW THE FLOW



If mailbox 90=4 and 91=7, the accumulator reaches 11 and mailbox 92 stores 11. This example is valid LMC.

EXAM-READY STATEMENT

Immediate/indexed/indirect examples are generic pseudo-assembly, NOT instructions you may assume exist in standard OCR LMC.

WORDS THAT EARN MARKS

ACC	Accumulator	**DAT**	Data mailbox
Direct	Read location	**Indirect**	Follow pointer

APPENDIX EXAMPLES / LMC appendix worked trace >

1.2.4(c,d) / STEP-BY-STEP WORKED EXAMPLE

Trace a complete LMC sequence

Follow each change in order. A model is only useful when you can explain both what changed and why.

TRACE / DECISION TABLE

STEP	VALUE OR ACTION	REASON / INTERPRETATION
Memory	90 DAT 4; 91 DAT 7; 92 DAT 0	These are data mailboxes, not literal operands.
LDA 90	ACC ← RAM[90] = 4	Load value, not the number 90.
ADD 91	ACC ← 4 + RAM[91] = 11	Arithmetic result retained in ACC.
STA 92	RAM[92] ← 11	Store result, ACC remains 11.
OUT	Output 11	Output current ACC.
HLT	Stop execution	No further instruction is executed.

TRANSFER IT TO AN EXAM ANSWER

Immediate/indexed/indirect examples are generic pseudo-assembly, NOT instructions you may assume exist in standard OCR LMC.

Exam habit: state the condition, describe the mechanism, and link it to the specific outcome. Check calculations and intermediate values, not just the final result.

1.2.4(c,d) / INDEPENDENT EXAM PRACTICE

Use the model without copying it

Questions are original teacher-written practice, not OCR past-paper questions. Work in black pen; check the next page only afterwards.

BLACK PEN: ANSWER FIRST. THEN CHECK THE SEPARATE MARK POINTS.

QUESTION 1 | 5 MARKS

Given 90 DAT 6, 91 DAT 2, 92 DAT 0, trace LDA 90, SUB 91, STA 92, OUT, HLT. Give ACC/output/mailbox 92.

QUESTION 2 | 6 MARKS

Explain immediate, direct, indirect and indexed addressing, using a simple example of each.

1.2.4(c,d) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct, explain

Check method and vocabulary, not just the final number. These are teacher-written points, not an OCR mark scheme.

Q1 / 5 MARKS

- After LDA 90: ACC = 6. After SUB 91: ACC = 4.
- STA 92 puts 4 in mailbox 92. OUT displays 4. HLT stops.

Q2 / 6 MARKS

- Immediate: operand is value in instruction, e.g. ADD #5 (generic).
- Direct: instruction gives location holding operand, e.g. ADD 90 means RAM[90].
- Indirect: instruction identifies a location containing an operand address/pointer.
- Indexed: effective address is base plus index/offset, e.g. ADD 90,X.
- Note only direct-style examples are in ordinary LMC syntax.

COMMON MISCONCEPTION

LMC has no standard LDI, indirect or indexed instruction in OCR's basic set. Do not write generic pseudo-assembly as an LMC answer.

Green pen: For each instruction, write "effective address" and "operand value" as two different entries.

1.2.4(e) / BUILD THE MENTAL MODEL

Encapsulation protects the state; polymorphism varies behaviour

A class defines structure and behaviour; an object is a runtime instance. Inheritance and polymorphism solve different design problems.

THREE PIECES OF THE IDEA

CLASS / OBJECT

A class is a blueprint; an instance holds attributes and exposes methods.

ENCAPSULATION

State and operations are grouped; access restrictions mediate valid changes.

INHERITANCE / POLYMORPHISM

Subclasses reuse/extend a base type; the same interface can select different behaviour.

MODEL / FOLLOW THE FLOW

Shape base class → **draw() method** → **Circle override** → **Square override**

Calling draw() on Shape references can dispatch to different implementations for Circle and Square objects.

EXAM-READY STATEMENT

Encapsulation means controlled access to object state, not merely placing public variables inside a class.

WORDS THAT EARN MARKS

- **Attribute** Stored property
- **Method** Object operation
- **Override** Replace method
- **Polymorphism** Same interface

APPENDIX EXAMPLES / OCR object notation >

1.2.4(e) / STEP-BY-STEP WORKED EXAMPLE

Model a safe bank account object

Follow each change in order. A model is only useful when you can explain both what changed and why.

TRACE / DECISION TABLE

STEP	VALUE OR ACTION	REASON / INTERPRETATION
Class	Account(balance)	Define state and methods.
Private	balance not directly writable	Other code cannot bypass validation.
Method	withdraw(amount)	Validate amount positive and not greater than balance.
Instance	a = new Account(40)	Object a starts with balance 40.
Call	a.withdraw(15)	Balance becomes 25 after validation.
Extension	SavingsAccount overrides interest()	An inherited type can specialise behaviour.

TRANSFER IT TO AN EXAM ANSWER

Encapsulation means controlled access to object state, not merely placing public variables inside a class.

Exam habit: state the condition, describe the mechanism, and link it to the specific outcome. Check calculations and intermediate values, not just the final result.

1.2.4(e) / INDEPENDENT EXAM PRACTICE

Use the model without copying it

Questions are original teacher-written practice, not OCR past-paper questions. Work in black pen; check the next page only afterwards.

BLACK PEN: ANSWER FIRST. THEN CHECK THE SEPARATE MARK POINTS.

QUESTION 1 | 6 MARKS

Explain classes, objects, attributes, methods and encapsulation using a bank-account example.

QUESTION 2 | 6 MARKS

Explain how inheritance and polymorphism could be used for a hierarchy of shapes with draw() methods.

1.2.4(e) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct, explain

Check method and vocabulary, not just the final number. These are teacher-written points, not an OCR mark scheme.

Q1 / 6 MARKS

- Class is template defining data and operations.
- Account object is an instance with balance attribute and withdraw method.
- Encapsulation combines state/behaviour and controls access so invalid withdrawals are prevented.
- Methods can enforce data invariants.

Q2 / 6 MARKS

- Base class Shape defines shared interface/behaviour.
- Circle/Square inherit and can add or override methods.
- Polymorphic call to draw() selects implementation appropriate to runtime object.
- This enables common client code and easier extension.

COMMON MISCONCEPTION

Inheritance shares/reuses a type relationship; polymorphism selects different behaviour through a common interface.

Green pen: Draw a class and two instances; label which features belong to the class and which values belong to each object.